Guión de trabajo tutorizado

Sesión práctica 1

Gestión y visualización de datos masivos

Profesor Dr. Carlos D. Barranco González
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA - Sesión Práctica 1 - Actividad.pdf» (Campus Virtual UPO).

Objetivos

• Se capaz de resolver casos prácticos relacionados con la gestión y visualización de datos masivos.

• Emplear herramientas adecuadas para la gestión de los datos.

• Ser capaz de implementar un proyecto básico de big data en las fases de extracción de datos y almacenamiento.

Actividades

A1. (60 minutos – 1 punto) Durante la sesión presencial se ha empleado la herramienta Talend Open Studio [1], con la que puede que no esté familiarizado. En esta actividad se preparará el entorno y se conseguirá su ejecución.

El uso de la herramienta es muy intuitivo, no obstante, y de modo opcional, podrá encontrar en su menú de ayuda un tutorial básico de su funcionalidad o, si desea profundizar más puede usar recursos online como [2].

Como prueba de realización de esta actividad el alumno mostrará capturas de pantalla, con su explicación correspondiente, ilustrando el proceso de puesta en marcha.

A2. (180 minutos – 3.4 puntos) El objetivo de esta actividad es que el alumno replique los pasos realizados por el profesor durante el caso práctico 1, que procesaba los datos de Wikipedia. Para ello, repase la grabación de la clase y realice usted mismo cada uno de los pasos. Tenga en cuenta que deberá hacer una instalación tanto de MongoDB como de un cliente para MongoDB (Studio 3T Community Edition).

Como resultado de la actividad, el alumno documentará todo el proceso, explicitando las actividades que se realizan y el sentido de las mismas. Todo ello quedará ilustrado con capturas de pantalla que deberán ser perfectamente contextualizadas con la explicaciones textuales adecuadas.

A3. (180 minutos – 3.4 puntos) En ésta actividad se replicará el caso práctico 2. Por tanto, se procederá de la misma forma para la actividad anterior y se generará un resultado similar. Tenga de nuevo en cuenta que necesitará desplegar un contenedor con HBase.

A4. (120 minutos – 2.2 puntos) En clase se han visto diferentes fuentes abiertas de datos. El objetivo de esta actividad es que el alumno consulte alguna de ellas, encuentre datos que le resulten de interés y plantee un caso práctico.

Como resultado de la actividad, el alumno deberá describir (por separado y de forma específica) la fuente de datos que ha encontrado, la/s razón/es de que ésta sea interesante, la posibilidad de análisis que ve en la misma, y un planteamiento esquemático de cómo podría realizar el análisis.

Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- Los alumnos tienen hasta el segundo lunes, desde la impartición de la clase, para devolver el cuestionario.

Bibliografía

1. Talend Open Studio (Se han de emplear las credenciales de su usuario en la UPO para acceder al enlace) https://upolavidemy.sharepoint.com/:u:/g/personal/cdbargon_upo_es/EbRZ5bxkqglLvkCjusbJc4BxgfHhgye4NBBMM4CnOgUAw?e=dFepQP

2. Curso completo de Talend - Talend Open Studio https://www.youtube.com/playlist?list=PLCGSGecP1ZlbiqQ4qSrFP-4mOBiK2EZU3

RESOLUCIÓN

Alcance de la entrega. La **A2** (caso 1, Wikipedia → MongoDB), la **A3** (caso 2, multas de Los Ángeles → HBase) y la **A4** están resueltas con ejecución real. La **A1** (Talend Open Studio) **no se ha realizado por limitación de hardware** del equipo disponible, como se justifica en su apartado. La carga que en clase hace Talend se ha sustituido por un script de Python (A2) y por la herramienta nativa `ImportTsv` de HBase (A3).

A1. Puesta en marcha de Talend Open Studio for Big Data

No ejecutada, por limitación de hardware. Las propias indicaciones de la sesión establecen que Talend Studio for Big Data necesita **al menos 8 GB de RAM (recomendado 16 GB) y de 2 a 4 núcleos**, con un JDK 11 y la instalación de dependencias en el primer arranque. La alternativa sería una máquina virtual Linux, que exigiría esa misma memoria además de la del sistema anfitrión. El equipo disponible **no alcanza esos requisitos**: con la memoria de la que dispone, Studio (un IDE basado en Eclipse) y una MV no pueden funcionar a la vez de forma estable. Por eso se ha decidido no hacer esta actividad. En la A2 y en la actividad de la Sesión Práctica 2, la carga que en clase se hace con el *job* de Talend se ha hecho con un script de Python equivalente (extracción XPath → documentos → MongoDB), que reproduce la misma lógica ETL con un consumo de memoria mínimo. Procedimiento que se seguiría con un equipo adecuado:

- Instalar un **JDK 11** (no JRE). En clase se usó `openjdk 11.0.19`; Azul Zulu es una buena opción. Comprobar con `java -version` y fijar `JAVA_HOME`.
- Descargar Talend Open Studio for Big Data **8.0.1** del enlace del curso (SharePoint UPO, con las credenciales de la universidad) y **descomprimirlo**: no hay instalador.
- Lanzar el ejecutable de la plataforma (`TOS_BD-linux-gtk-x86_64` en Linux, `TOS_BD-win-x86_64.exe` en Windows). Se recomienda Linux, con 8 GB de RAM como mínimo (mejor 16 GB) y 2-4 núcleos.
- En el primer arranque, aceptar la licencia, crear un proyecto local y dejar que instale las **dependencias obligatorias**. Para ahorrar tiempo, no marcar las adicionales.
- Verificar el entorno creando un *Job* sencillo (`tFileInputDelimited` → `tLogRow`) y ejecutándolo desde la pestaña *Run*.

A2. Caso práctico 1: top 10 de contribuidores de la Wikipedia

Objetivo: determinar los 10 usuarios que más contribuyen a una Wikipedia pequeña (la **extremeña**, *Güiquipeya*) y representarlo en un gráfico. El flujo sigue las fases del proyecto Big Data visto en clase:

Fase	En clase	En esta resolución	Por qué
Adquisición	Dump XML de dumps.wikimedia.org	Igual: `extwiki-latest-pages-articles.xml.bz2` (dump del 01/09/2026, 6 MB, 7.697 páginas)	Fuente abierta y semiestructurada
Carga (ETL)	Job de Talend (XPath → MongoDB)	Script Python equivalente (`etl_wikipedia.py`): XPath/ElementTree →	Talend no disponible (véase la nota)

		documentos → `insertMany`	
Almacenamiento	MongoDB	MongoDB 8.3.11, base `wikipedia`, colección `articles`	Datos semiestructurados → BD orientada a documentos
Consulta	`mongo --quiet wikipedia vecesAutor.js > result.json`	Igual con `mongosh` (el shell `mongo` ya no existe desde MongoDB 6)	—
Cliente	Studio 3T Community	`mongosh`	Entorno sin interfaz gráfica
Visualización	RAWGraphs	Gráfico de barras generado con Python/matplotlib	Mismo tipo de gráfico

Extracción y carga

El *job* de Talend recorre cada elemento `<page>` del XML, extrae con XPath los campos de interés y los inserta como documentos en MongoDB. El script hace exactamente eso, ***en streaming***: no carga todo el XML en memoria. Además convierte la fecha a tipo `Date` y registra si la página es una redirección:

```
#!/usr/bin/env python3
"""ETL del caso práctico 1 (sustituye al job de Talend):
Extract : lee el dump XML de la Wikipedia en extremeño (pages-articles.xml.bz2) en
streaming.
Transform: aplana cada <page> a un documento JSON (mismos campos que el job de clase:
title, ns, id,
           revisionid, timestamp, contributorusername, contributorid, textbytes).
Load    : inserta en MongoDB, base de datos 'wikipedia', colección 'articles' (insertMany
por lotes)."""
import bz2, sys, xml.etree.ElementTree as ET
from datetime import datetime
from pymongo import MongoClient

src = sys.argv[1]
NS = '{http://www.mediawiki.org/xml/export-0.11/}'
col = MongoClient('mongodb://localhost:27017')['wikipedia']['articles']
col.drop()
lote, total = [], 0
for ev, el in ET.iterparse(bz2.open(src), events=('end',)):
    if el.tag != NS + 'page':
        continue
    rev = el.find(NS + 'revision'); con = rev.find(NS + 'contributor'); txt = rev.find(NS +
'text')
    doc = {
        'title': el.findtext(NS + 'title'),
        'ns': int(el.findtext(NS + 'ns')),
        'id': int(el.findtext(NS + 'id')),
        'redirect': el.find(NS + 'redirect') is not None,
        'revisionid': int(rev.findtext(NS + 'id')),
        'timestamp': datetime.fromisoformat(rev.findtext(NS + 'timestamp').replace('Z',
'+00:00')),
        'contributorusername': con.findtext(NS + 'username') or con.findtext(NS + 'ip'),
        'contributorid': int(con.findtext(NS + 'id')) if con.findtext(NS + 'id') else None,
        'textbytes': int(txt.get('bytes', 0)) if txt is not None else 0,
    }
    lote.append(doc); total += 1
    if len(lote) == 1000:
        col.insert_many(lote); lote = []
    el.clear()
if lote: col.insert_many(lote)
print(f'Documentos cargados en wikipedia.articles: {total}')
```

```
aramber@localhost:~$ ls -lh extwiki-latest-pages-articles.xml.bz2
-r--r--r-- 1 root root 6.0M Sep 19 12:38 extwiki-latest-pages-articles.xml.bz2
aramber@localhost:~$ bzcat extwiki-latest-pages-articles.xml.bz2 | grep -c '<page>'
7697
aramber@localhost:~$ time python3 etl_wikipedia.py extwiki-latest-pages-articles.xml.bz2
Documentos cargados en wikipedia.articles: 7697

real    0m2.259s
user    0m1.738s
sys     0m0.064s
aramber@localhost:~$ mongosh --quiet wikipedia --eval 'db.articles.countDocuments()'
7697
aramber@localhost:~$ mongosh --quiet wikipedia --eval 'db.articles.findOne({ns:0, redirect:false})'
{
  _id: ObjectId('6aae661a15f10d035a30897b'),
  title: 'Portá antigua',
  ns: 0,
  id: 246,
  redirect: false,
  revisionid: 124852,
  timestamp: ISODate('2023-05-27T02:37:19.000Z'),
  contributorusername: 'Sa~eswiki',
  contributorid: 8342,
  textbytes: 9440
}
```

Figura 1. Dump descargado (7.697 páginas), ejecución del ETL, recuento en MongoDB y ejemplo de documento cargado.

Cada documento representa una página con los datos de su **última revisión**, que es lo que contiene el volcado *pages-articles*. `contributorusername` es, por tanto, el último editor de cada página, igual que en el caso visto en clase.

Consulta y resultado

La consulta del material suplementario agrupa por `contributorusername`, cuenta los documentos de cada usuario, ordena de mayor a menor y se queda con los 10 primeros. `printjson` y `toArray` vuelcan el resultado en formato JSON a `result.json`:

```
aramber@localhost:~$ cat vecesAutor.js
printjson(db.articles.aggregate(
  [
    { $group: { _id: "$contributorusername", count: { $sum: 1 } } },
    { $sort: { count: -1 } },
    { $limit: 10 }
  ]
).toArray())
aramber@localhost:~$ mongosh --quiet wikipedia vecesAutor.js > result.json

aramber@localhost:~$ cat result.json
[
  {
    _id: 'EN-Jungwon',
    count: 1063
  },
  {
    _id: 'Sa~eswiki',
    count: 1008
  },
  {
    _id: 'InternetArchiveBot',
    count: 615
  },
  {
    _id: 'HackerPunki',
    count: 482
  },
  {
    _id: 'Caro de Segeda',
    count: 479
  },
  {
    _id: 'El tiu Cancho',
    count: 351
  },
  {
    _id: 'Althair',
    count: 344
  },
  {
    _id: 'Tiu Fraili',
    count: 224
  },
  {
    _id: 'Sa',
    count: 203
  },
  {
    _id: 'Olarcos',
    count: 201
  }
]
```

Figura 2. Script `vecesAutor.js`, ejecución con salida a fichero y contenido de `result.json`.

#	Usuario	Páginas
1	EN-Jungwon	1.063
2	Sa~eswiki	1.008
3	InternetArchiveBot	615
4	HackerPunki	482
5	Caro de Segeda	479
6	El tiu Cancho	351
7	Althair	344
8	Tiu Fraili	224
9	Sa	203
10	Olarcos	201

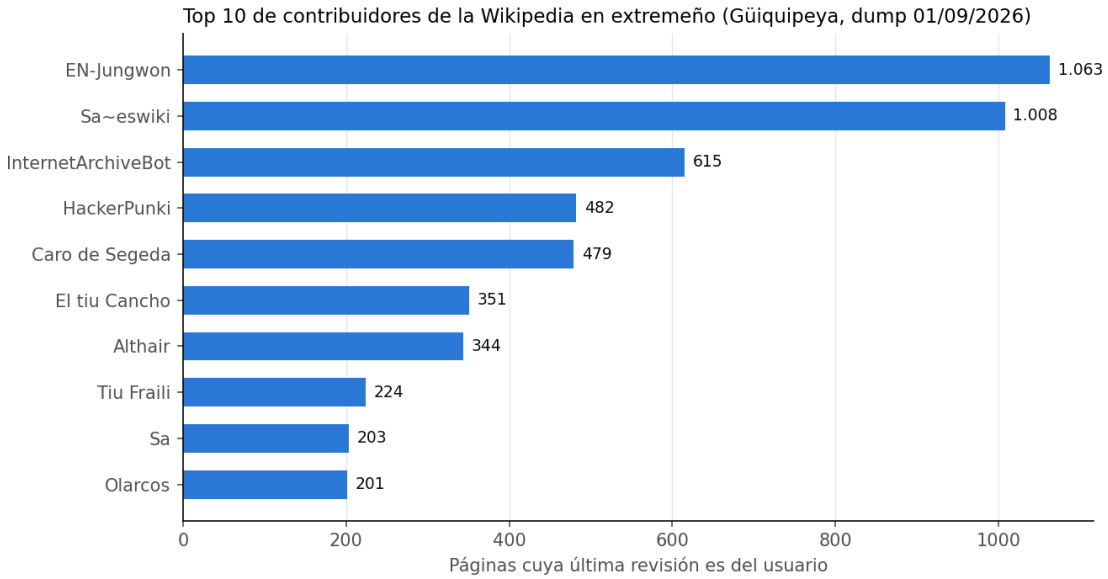


Figura 3. Top 10 de contribuidores (último editor de cada página).

Interpretación. Dos usuarios destacan con más de 1.000 páginas cada uno (EN-Jungwon y Sa~eswiki). También aparece un **bot** (*InternetArchiveBot*, tercero con 615), que corrige enlaces rotos de forma masiva. Es un aviso típico de «garbage in – garbage out»: si se quieren contribuidores humanos hay que filtrar los bots, por nombre o con la lista de usuarios con el grupo *bot*. Hay que recordar también que la métrica cuenta al **último editor**, no el número total de ediciones. Para medir contribuciones reales habría que cargar el volcado con historial completo (`pages-meta-history`, 40 MB comprimido) y contar revisiones por usuario. También sería razonable limitarse a artículos (`ns = 0`) y excluir redirecciones.

A3. Caso práctico 2: multas de aparcamiento de Los Ángeles en HBase

Objetivo: calcular cuánto se ha recaudado con las multas de una muestra de 100.000 registros del conjunto *Los Angeles Parking Citations*. **Decisiones técnicas:** la fuente es un CSV con 22 atributos del que sólo interesa uno (el importe, `Fine amount`), así que encaja con una **base de datos columnar (HBase)**: se lee sólo la columna necesaria. HBase no agrega datos sin programar, así que se exporta la columna a un fichero y se suma con **AWK**.

Adquisición y muestra

El conjunto se descarga de Kaggle (*cityofLA/los-angeles-parking-citations*, *Download dataset as zip*, 413 MB) con la sesión iniciada:

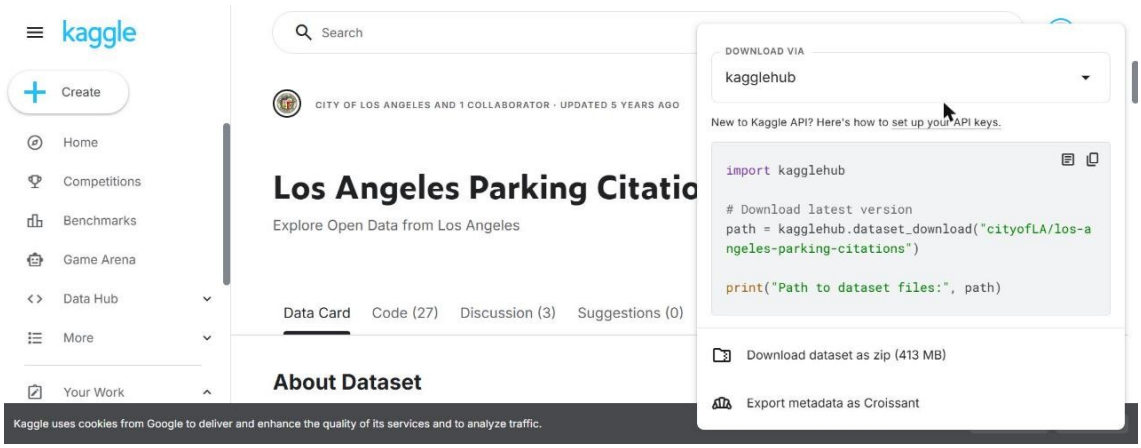


Figura 4. Ficha del conjunto en Kaggle y menú de descarga (413 MB comprimido).

El ZIP contiene `parking-citations.csv` de **2,04 GB**. En lugar de descomprimirlo, un script lo lee en *streaming*, cuenta las líneas y guarda las 100.000 primeras (cabecera incluida), que es lo que hace `head -n 100000` en el guion de clase:

```
# Cuenta las líneas de parking-citations.csv leyendo el ZIP en streaming
# (sin descomprimir 2 GB a disco) y guarda las 100.000 primeras (cabecera incluida)
import zipfile, time
t = time.time()
z = zipfile.ZipFile('archive.zip')
n = 0
with z.open('parking-citations.csv') as f, open('parking-small.csv', 'wb') as o:
    for line in f:
        if n < 100000:
            o.write(line)
        n += 1
print('lineas', n, 'seg', round(time.time() - t, 1))
```

```
aramber@localhost:~$ ls -l archive.zip        # descargado de Kaggle (cityofLA/los-angeles-parking-citations)
-rwx------ 1 aramber aramber 413089194 Sep 19 14:03 archive.zip
aramber@localhost:~$ python3 -c "import zipfile; [print(i.filename, i.file_size) for i in zipfile.ZipFile('archive.zip').infolist()]"
LADOT-Xerox Crib Sheet Agency Codes 12-31-2015 (1).pdf 194805
parking-citations.csv 2039199744
socrata_metadata.json 28322
aramber@localhost:~$ python3 cuenta_y_muestra.py   # cuenta las líneas del CSV (sin descomprimirlo a disco) y guarda las 100.000 primeras
lineas 13077725 seg 61.1
aramber@localhost:~$ head -3 parking-small.csv
Ticket number,Issue Date,Issue time,Meter Id,Marked Time,RP State Plate,Plate Expiry Date,VIN,Make,Body Style,Color,Lo
cation,Route,Agency,Violation code,Violation Description,Fine amount,Latitude,Longitude,Agency Description,Color Descr
iption,Body Style Description
1103341116,2015-12-21T00:00:00.000,1251,,,CA,200304,,HOND,PA,GY,13147 WELBY WAY,01521,1,4000A1,NO EVIDENCE OF REG,50,9
9999,99999,,,
1103700150,2015-12-21T00:00:00.000,1435,,,CA,201512,,GMC,VN,WH,525 S MAIN ST,1C51,1,4000A1,NO EVIDENCE OF REG,50,99999
,99999,,,
aramber@localhost:~$ ls -l parking-small.csv
-rwx------ 1 aramber aramber 13940519 Sep 19  2026 parking-small.csv
```

Figura 5. Contenido del ZIP, recuento de líneas del CSV completo y cabecera de la muestra. Ejecutado en el equipo local.

El fichero completo tiene **13.077.725 líneas**, es decir, **13.077.724 multas** más la cabecera. La muestra tiene 99.999 multas. La versión actual del conjunto tiene **22 columnas**: las 19 del guion más tres descriptivas añadidas por el portal (`Agency Description`, `Color Description`, `Body Style Description`).

Carga en HBase

Se usa HBase 2.6 en modo pseudodistribuido, el mismo entorno que en la actividad de casos prácticos de BD masivamente paralelas. Equivale al contenedor `cbarrancoupo/hbase-`

`pseudo` del curso. La tabla `citations` tiene una sola familia `c`, y la clave de fila es el número de multa (`Ticket number`). En clase la carga la hace un *job* de Talend (`tHBaseOutput`). Aquí se usa la herramienta nativa **`ImportTsv`** (MapReduce en modo local). Antes se convierte el CSV a TSV con el módulo `csv` de Python para no romper los campos entrecomillados que contienen comas:

```
python3 -c "import csv; w=csv.writer(open('parking-small.tsv','w'),delimiter='\
t',lineterminator='\n'); r=csv.reader(open('parking-small.csv')); next(r); [w.writerow(x) for x in
r]"
echo "create 'citations', 'c'" | hbase shell -n
hbase org.apache.hadoop.hbase.mapreduce.ImportTsv \
  -
Dimporttsv.columns=HBASE_ROW_KEY,c:issue_date,c:issue_time,c:meter_id,c:marked_time,c:rp_state_pla
te,c:plate_expiry_date,c:vin,c:make,c:body_style,c:color,c:location,c:route,c:agency,c:violation_c
ode,c:violation_description,c:fine_amount,c:latitude,c:longitude,c:agency_desc,c:color_desc,c:body
_style_desc \
  citations file:///data/parking-small.tsv
```

```
aramber@localhost:~$ echo "create 'citations', 'c'" | hbase shell -n 2>/dev/null
hbase:001:0> create 'citations', 'c'
Created table citations
Took 1.0553 seconds
=> Hbase::Table - citations
hbase:002:0>
aramber@localhost:~$ hbase org.apache.hadoop.hbase.mapreduce.ImportTsv -Dimporttsv.columns=HBASE_ROW_KEY,c:issue_date,c:issue_time,c:mete
r_id,c:marked_time,c:rp_state_plate,c:plate_expiry_date,c:vin,c:make,c:body_style,c:color,c:location,c:route,c:agency,
c:violation_code,c:violation_description,c:fine_amount,c:latitude,c:longitude,c:agency_desc,c:color_desc,c:body_style_
desc citations file:///data/parking-small.tsv 2>&1 | grep -E 'Job .* completed|Bad Lines|Map input records|Map output
records|ImportTsv'
        Map input records=99999
        Map output records=99999
    ImportTsv
        Bad Lines=0
2026-09-19T16:21:36,529 INFO  [main] mapreduce.Job: Job job_local1930959761_0001 completed successfully
        Map input records=99999
        Map output records=99999
    ImportTsv
        Bad Lines=0
```

Figura 6. Creación de la tabla y carga con ImportTsv: 99.999 registros leídos y escritos, 0 líneas erróneas.

Consulta, exportación y suma

```
aramber@localhost:~$ echo "count 'citations', INTERVAL => 50000" | hbase shell -n 2>/dev/null | grep -v '^hbase'
Current count: 50000, row: 4274073215
99999 row(s)
Took 1.5634 seconds
=> 99999
aramber@localhost:~$ echo "scan 'citations', {COLUMNS => ['c:fine_amount', 'c:violation_description'], LIMIT => 3}" | hbase shell -n 2>/d
ev/null | grep -v '^hbase'
ROW  COLUMN+CELL
 1001235955 column=c:fine_amount, timestamp=2026-09-19T16:21:26.534, value=68
 1001235955 column=c:violation_description, timestamp=2026-09-19T16:21:26.534, value=BLOCKING DRIVEWAY
 1004848515 column=c:fine_amount, timestamp=2026-09-19T16:21:26.534, value=25
 1004848515 column=c:violation_description, timestamp=2026-09-19T16:21:26.534, value=EXPIRED TAGS
 1004848526 column=c:fine_amount, timestamp=2026-09-19T16:21:26.534, value=25
 1004848526 column=c:violation_description, timestamp=2026-09-19T16:21:26.534, value=EXPIRED TAGS
3 row(s)
Took 0.4332 seconds
aramber@localhost:~$ echo "scan 'citations', {COLUMNS => ['c:fine_amount']}" | hbase shell -n 2>/dev/null | grep -o 'value=[0-9.]*' | cut
 -d= -f2 > /data/out.csv

aramber@localhost:~$ wc -l /data/out.csv; head -5 /data/out.csv; grep -c '^$' /data/out.csv
99999 /data/out.csv
68
25
25
50
363
131
aramber@localhost:~$ awk 'NF { sum += $1; n++ } END { printf "Total recaudado: %.2f USD en %d multas con importe (media %.2f USD)\n", sum
, n, sum/n }' /data/out.csv
Total recaudado: 7049049.00 USD en 99868 multas con importe (media 70.58 USD)
```

Figura 7. Recuento de la tabla, scan de ejemplo, exportación de la columna c:fine_amount a out.csv y suma con AWK.

Resultado	Valor
Filas en la tabla `citations`	99.999

Valores exportados a `out.csv`	99.999 (131 vacíos: multas sin importe)
Multas con importe	99.868
Total recaudado en la muestra	**7.049.049 USD**
Importe medio	70,58 USD

Comentarios. (1) Las 131 multas sin importe exportan una línea vacía. Un AWK que sólo sume no se ve afectado, pero **sí falsearía la media** si se divide entre todas las líneas (70,49 en vez de 70,58 USD). Por eso el AWK filtra con `NF`. (2) La suma se ha contrastado leyendo el CSV original con Python: sale el mismo total, 7.049.049 USD. (3) Las coordenadas `99999` son un código de «sin geolocalizar», no una posición válida. Habría que tratarlas como nulas si se quisieran usar. (4) El orden de las columnas de `ImportTsv` debe coincidir con la cabecera del CSV. Con el dataset actual hay que añadir las tres columnas descriptivas o se rechazarían las líneas. (5) En un clúster real la suma se haría sin exportar, con un *job* MapReduce/Spark o un coprocesador de agregación, pero la exportación con `scan` + AWK es la que plantea el caso.

A4. Propuesta de caso práctico con datos abiertos

Fuente de datos

REData API de Red Eléctrica (REE), https://apidatos.ree.es. Es el servicio de datos abiertos del operador del sistema eléctrico español. Es una API REST pública, **sin token**, que devuelve JSON (estándar JSON:API) con el patrón `GET /{es|en}/datos/{categoría}/{widget}?start_date=…&end_date=… &time_trunc=hour|day|month|year`, y filtros geográficos opcionales (peninsular, Baleares, Canarias o comunidad autónoma). Ofrece seis categorías: **balance, demanda, generación, intercambios, transporte y mercados** (precios). Por ejemplo, `/es/datos/generacion/estructura-generacion` devuelve la generación por tecnología y `/es/datos/mercados/precios-mercados-tiempo-real` el precio horario del mercado y del PVPC.

Por qué es interesante

- **Relevancia:** el precio de la electricidad y la penetración renovable afectan a hogares y empresas, y los datos son oficiales y horarios, lo que permite análisis finos.
- **Volumen y velocidad:** una década de datos horarios por tecnología, precio y zona supone millones de registros y se actualiza continuamente. Cumple varias de las «V» del Big Data.
- **Calidad y acceso:** fuente oficial, documentada, gratuita y sin registro, en un formato (JSON) fácil de cargar en una base de datos documental o columnar.

Posibilidades de análisis

- **Efecto de las renovables en el precio** (*merit order*): relación entre el porcentaje horario de solar y eólica y el precio del mercado; horas de precio cero o negativo.
- **Patrones de demanda:** estacionalidad diaria, semanal y anual; efecto de la temperatura o de los festivos; comparación entre comunidades autónomas.

- **Predicción** del precio o la demanda del día siguiente a partir de la previsión de renovables y del histórico, útil para planificar consumos o baterías.
- **Asesoramiento energético:** con el precio horario se puede calcular qué tarifa (PVPC o precio fijo) o qué franja de consumo conviene a un perfil de cliente.

Planteamiento esquemático del análisis

- **Adquisición:** script (o *job* de Talend con `tREST`) que recorre la API por meses (la API limita el rango por petición) para demanda, estructura de generación y precios con `time_trunc=hour`, desde 2015.
- **Almacenamiento:** respuesta JSON cruda en **MongoDB** (*data lake* de documentos, un documento por petición) y, tras transformarla, una **tabla columnar** (HBase o BigQuery) con clave `fecha-hora + zona` y columnas por tecnología, demanda y precio.
- **Transformación:** aplanar los *arrays* de valores, unificar zonas horarias y el cambio de hora, rellenar huecos y calcular el porcentaje renovable por hora.
- **Análisis:** correlación y regresión precio ~ % renovable + demanda + hora + mes; modelo de predicción (p. ej. *gradient boosting* o BigQuery ML) validado con los últimos meses.
- **Visualización:** cuadro de mando (Looker Studio o Power BI) con evolución del precio, *mix* de generación y un mapa de calor hora × mes del precio medio.

Referencias

[1] Wikimedia Downloads – extwiki. https://dumps.wikimedia.org/extwiki/latest/

[2] Material suplementario de la Sesión Práctica 1 (consulta vecesAutor.js y comandos del caso 2).

[3] Red Eléctrica – REData API. https://www.ree.es/es/datos/apidatos

[4] Apache HBase Reference Guide – ImportTsv. https://hbase.apache.org/book.html#importtsv